

DBMS Indexing

1. What is Indexing?

Definition

Indexing is a technique used in DBMS to speed up data retrieval operations.

An index works like:

- Book index
- Phone contact list
- Dictionary arrangement

Instead of searching entire table row by row, database uses index to directly locate data.

Real-Life Example

Suppose a book has 1000 pages.

Without index:

- Need to search page by page.

With index:

- Directly jump to required topic page number.

Same concept used in databases.

2. Why Indexing is Needed?

Without indexing:

- Database performs Full Table Scan
- Search becomes slow for large tables

Indexing improves:

- Search speed
 - Query performance
 - Data retrieval efficiency
-

Example Without Index

RollNo Name

1	Rahul
2	Aman
3	Priya
4	Simran

Finding RollNo = 4:

Database checks rows one by one.

Time Complexity:

$O(n)$

Example With Index

Database directly jumps to RollNo = 4.

Time Complexity approximately:

$O(\log n)$

3. How Indexing Works

Database creates separate structure:

Key → Address

Example:

RollNo Address

1	Row1
2	Row2
3	Row3

Now searching becomes fast.

4. Advantages of Indexing

- Faster searching
- Faster sorting
- Faster filtering
- Improves query performance

- Reduces disk access
-

5. Disadvantages of Indexing

- Extra storage required
 - Insert/Delete becomes slower
 - Index maintenance needed
-

6. Types of Indexing

Main types:

1. Primary Index
 2. Secondary Index
 3. Clustered Index
 4. Non-Clustered Index
 5. Dense Index
 6. Sparse Index
 7. Multilevel Index
 8. B-Tree Index
 9. Hash Index
-

7. Primary Index

Definition

Index created on primary key.

Data stored in sorted order.

Example

RollNo Name

- | | |
|---|-------|
| 1 | Rahul |
| 2 | Aman |
| 3 | Priya |

Primary index built on RollNo.

Characteristics

- Based on primary key
 - Unique values
 - Fast searching
-

8. Secondary Index

Definition

Index created on non-primary key attribute.

Example

RollNo Name

1	Rahul
2	Aman
3	Rahul

Index on Name.

Characteristics

- Non-unique values allowed
 - Multiple secondary indexes possible
-

9. Clustered Index

Definition

Data rows physically stored in same order as index.

Only ONE clustered index possible.

Example

Table stored sorted by RollNo.

1
2
3

4

5

Physical storage follows index order.

Advantages

- Very fast range queries
 - Faster retrieval
-

Disadvantages

- Insertions slower
 - Only one clustered index allowed
-

10. Non-Clustered Index

Definition

Data rows NOT physically stored in index order.

Separate index structure maintained.

Example

Index stores pointers:

Name Pointer

Rahul Row5

Aman Row2

Characteristics

- Multiple allowed
 - Extra storage needed
-

Difference Between Clustered and Non-Clustered Index

Clustered	Non-Clustered
------------------	----------------------

Physical order same	Physical order different
---------------------	--------------------------

Clustered	Non-Clustered
One only	Multiple allowed
Faster range queries	Slightly slower

11. Dense Index

Definition

Index entry exists for every search key value.

Example

RollNo Pointer

1	Row1
2	Row2
3	Row3

Every record has index entry.

Advantages

- Faster searching
-

Disadvantages

- More storage required
-

12. Sparse Index

Definition

Index entries created for only some records.

Example

RollNo Pointer

1	Block1
5	Block2

Not every row indexed.

Advantages

- Less storage
-

Disadvantages

- Slightly slower searching
-

Difference Between Dense and Sparse Index

Dense Index	Sparse Index
Entry for every record	Entry for some records
Faster	Slower
More storage	Less storage

13. Multilevel Index

Definition

Index on top of another index.

Used when index becomes very large.

Example

Level 1 Index
Level 2 Index
Actual Data

Advantages

- Faster searching
 - Reduced disk access
-

14. B-Tree Index

Definition

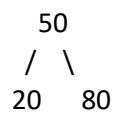
Balanced tree structure used for indexing.

Most commonly used indexing method.

Properties

- Balanced tree
 - Sorted data
 - Fast insertion/deletion/search
-

Example Structure



Time Complexity

Search = $O(\log n)$

Insert = $O(\log n)$

Delete = $O(\log n)$

Advantages

- Self-balancing
 - Efficient for large databases
-

15. B+ Tree Index

Definition

Advanced version of B-Tree.

Actual data stored only in leaf nodes.

Advantages

- Faster range queries
 - Better sequential access
-

Difference Between B-Tree and B+ Tree

B-Tree**B+ Tree**

Data in all nodes

Data only in leaf nodes

Slower range query

Faster range query

16. Hash Indexing**Definition**

Uses hash function to determine storage location.

Example

Hash(101) → Bucket 5

Advantages

- Very fast equality search
-

Disadvantages

- Poor range query performance
-

17. Ordered Indexing**Definition**

Index entries sorted according to key values.

Example

1 → Row1

2 → Row2

3 → Row3

18. Composite Index**Definition**

Index created on multiple columns together.

Example

INDEX(Name, Age)

Useful when queries use multiple columns.

19. Unique Index

Definition

Does not allow duplicate values.

Example

```
CREATE UNIQUE INDEX idx  
ON Student(Email);
```

20. SQL Syntax for Indexing

Create Index

```
CREATE INDEX idx_name  
ON Student(Name);
```

Create Unique Index

```
CREATE UNIQUE INDEX idx_email  
ON Student(Email);
```

Drop Index

```
DROP INDEX idx_name;
```

21. When Index Should Be Used?

Use indexing on:

- Frequently searched columns
 - Primary keys
 - Foreign keys
 - WHERE clause columns
 - JOIN columns
-

Avoid Indexing On

- Small tables
 - Frequently updated columns
 - Columns with low uniqueness
-

22. Indexing vs Hashing

Indexing	Hashing
Supports range queries	Does not support range queries
Ordered	Unordered
Uses trees	Uses hash function

23. Query Optimization Using Index

Without index:

```
SELECT * FROM Student  
WHERE RollNo = 100;
```

Database scans entire table.

With index:

Direct access happens.

24. Important Interview Questions

Q1. What is indexing?

Technique used to improve search performance.

Q2. Difference between clustered and non-clustered index?

Clustered changes physical order.

Non-clustered uses separate structure.

Q3. Difference between dense and sparse index?

Dense → every record indexed.

Sparse → some records indexed.

Q4. Why B+ Tree preferred in DBMS?

- Faster range queries
 - Better disk access
 - Balanced structure
-

Q5. Why indexing improves performance?

Because full table scanning avoided.

25. Shortcut Tricks for Exams

Primary Index

Primary key based.

Secondary Index

Non-primary attribute based.

Clustered Index

Physical order same.

Non-Clustered Index

Separate pointer structure.

Dense Index

Every record indexed.

Sparse Index

Some records indexed.